

DESIGN & ARCHITECTURE DOCUMENT

Interacting Tool

A top-down 2D interaction sandbox — how the systems fit together and who talks to whom

Unity 2D · URP

New Input System

DOTween

C# async / await

~125 scripts · 6.7k lines

namespace **IT.***

► Five design pillars (you'll see these everywhere)

1 State pattern

The player is always in exactly one `PlayerStateBase`. Each state owns its input handling and animations.

2 Interactable polymorphism

Every world object answers `Kind`, `Interact()`, `Release()`. The state machine never knows concrete types.

3 InteractionContext façade

Objects never see the player class — only a narrow interface: transform, look direction, inventory, animator, and an “I’m done” callback.

4 Observer / dependency system

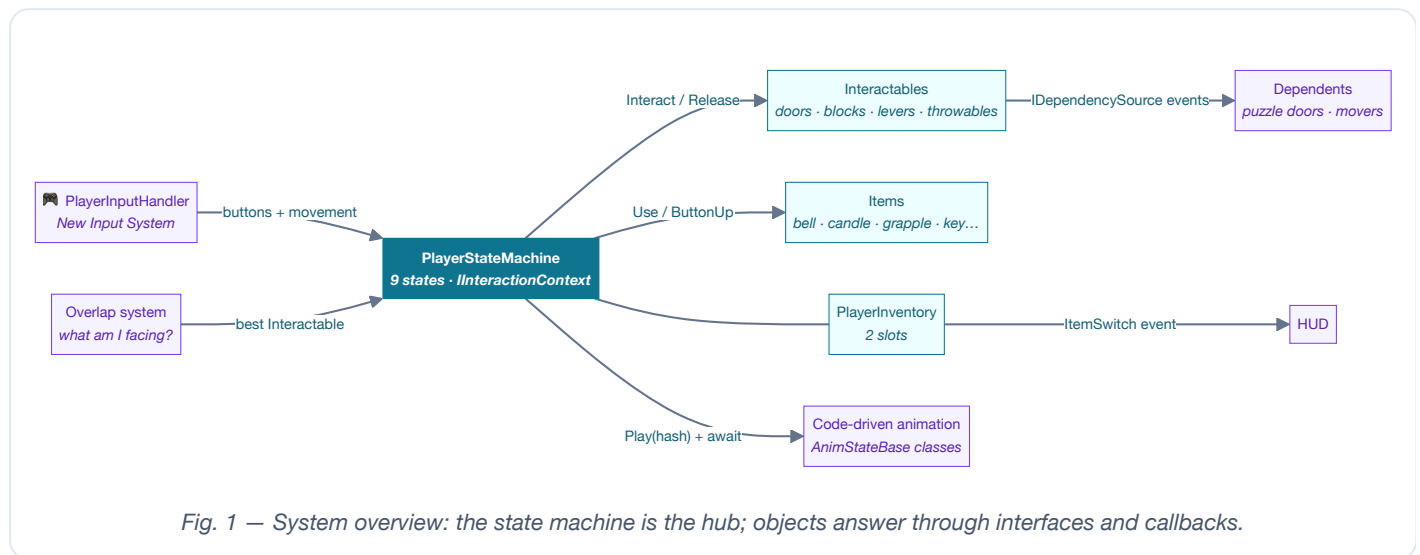
Puzzle outputs (lever pulled, block seated) publish via `IDependencySource<T>`; doors and platforms subscribe. Pieces never reference each other.

5 Physics layers as game state

An object's layer (`Interactable`, `Locked`, `HookSocketUsed` ...) encodes what can currently be done to it.

1 The Big Picture

Everything flows through the **player state machine**. Input drives it, it dispatches to **Interactables** (objects in the world) or **Items** (objects in the inventory), and those objects report back through callbacks and events when they finish.



Folder	Namespace	Role
Core/	IT.Core.*	Dependency system, InterfaceRef , damage interfaces, utilities, layer registry
Player/	IT.Player.*	Input, state machine, inventory, status/health, HUD
Interactables/	IT.Interactables.*	World objects the Interact button works on
Items/	IT.Items.*	Inventory items the Use button works on
Overlap/	IT.Overlap	Spatial queries — “what is the player facing / standing on?”
Animation/	IT.Animation.*	Code-driven animation (play Animator states by hash, await durations)
Effects/ · Enemies/	IT.Effects · IT.Enemies	Sprite flash effects · EnemyDummy test target
Editor/ · Prototype/	—	Inspector drawers & validators · apple FSM learning prototype

2 Player System

► Input routing — `PlayerInputHandler`

Input	Routed to
Movement (each <i>FixedUpdate</i>)	<code>stateMachine.UpdateMove(vector)</code>
Interact pressed	<code>stateMachine.Interact()</code> — find target, dispatch by <code>Kind</code>
Interact released	Current state is <code>IButtonUp</code> (pull) → <code>ButtonUp()</code> ; move-item state → <code>Release()</code>
UseItem pressed	<code>stateMachine.UseItem()</code> (ignored while a hold-style item is active)
UseItem released	<code>IButtonUp.ButtonUp()</code> — candle off, grapple retract
SwitchItem	<code>inventory.SwitchItem()</code>

► The state machine

`PlayerStateMachine` pre-allocates one instance of every state, holds the shared references (`rb`, `movement`, `animator`, `inventory`, `targeted item`), and exposes the `IInteractionContext` façade. `EndInteraction(next)` is how an item hands control back: `null` → idle; an `Interactable` → straight into the throw state carrying it.

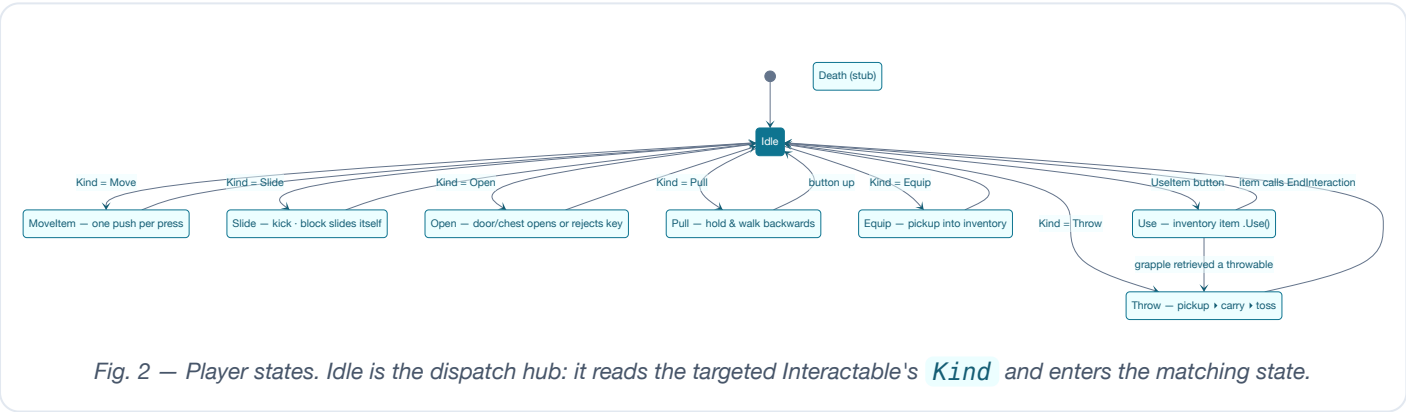


Fig. 2 — Player states. Idle is the dispatch hub: it reads the targeted `Interactable`'s `Kind` and enters the matching state.

PlayerIdleState

Default. Moves the rigidbody, tracks 4-way `LookDirection`, plays walk/stand clips, dispatches on `Kind`.

PlayerThrowState

Pick-up anim → `Interact()` parents the throwable to the player → carry-walk → press again: `Release()` tosses it, throw anim, idle.

PlayerPullState

Hold-style. Each `FixedUpdate` calls `Pull(delta)` and moves the player backward by what the handle *actually* moved — rope stays taut. Release latches or retracts the lever.

PlayerUseState

Calls `item.Use(context)`; the `item` ends the interaction via `EndInteraction`. Stores the item's `ButtonUp` if it's hold-style. Walking allowed if `CanWalk`.

PlayerSlideState / MoveItemState / OpenState

Thin: call `Interact()` , play success/failure anims (kick vs. hurt-toe), return to idle. The object owns its own motion.

Inventory & HUD

2-slot `PlayerInventory` ; full pickups swap into the world holder or eject via `ItemDropper` . `ItemSwitch` / health / lives events feed `PlayerStatusHUD` — UI never polls.

3 Interaction System

Every world object extends `Interactable` and answers three questions: **what am I** (`Kind`), **try it** (`Interact` → `bool`), and **second phase** (`Release`).

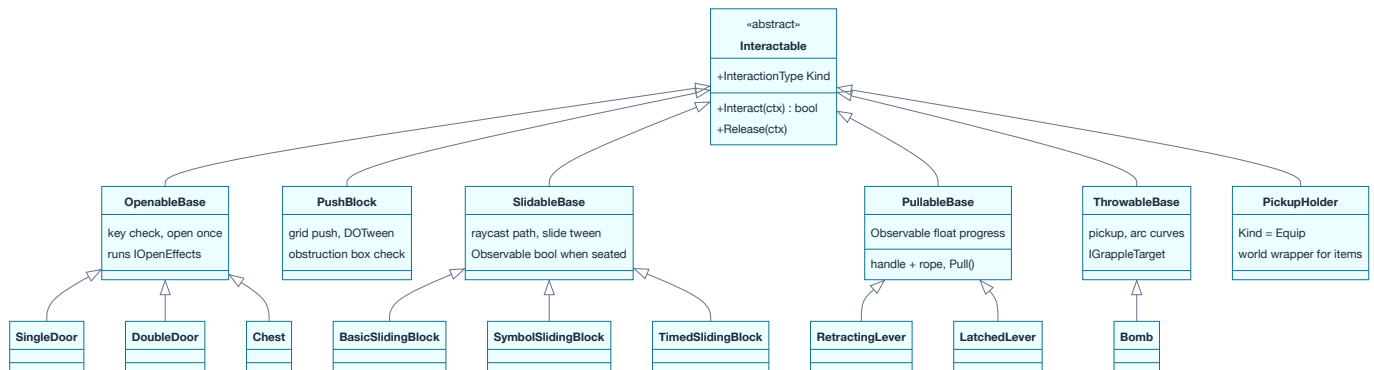


Fig. 3 — The `Interactable` family tree. Leaf classes are mostly thin: they pick a key type, a symbol, or an animation.

Openables — doors & chests

`OpenableBase` : optional `KeyTypes` check against the current inventory item (consumes it), open-once, then runs every `IOpenEffect` on the `GameObject`.

`SpawnItemOnOpen` is the chest payload. Effects are composable.

PushBlock

One grid cell per press: wind-up wiggle → eased move (mid-flight re-check snaps back if blocked) → settle. If seated ≥60% on a matching `KeyPort`, the script destroys itself — the block becomes scenery.

Sliding blocks

Kick → 3-point edge raycast finds the nearest obstruction → tween to just short of it. While sliding, spawns a `DamageOverlap` in front and a `KeyPortOverlap` underneath. Seated on a matching port → `Observable<bool>` flips true. Variants: symbol-matched, timed.

Throwables & bombs

Arc = an `AnimationCurve` driving a child sprite's local Y while the body moves; distance is the curve's last key. `Bomb` chains bounce curves with decaying speed; `BombTimer` runs an async 10s fuse → explosion prefab → splash damage to `IDamageable` s.

► Pullables — the dependency system end-to-end

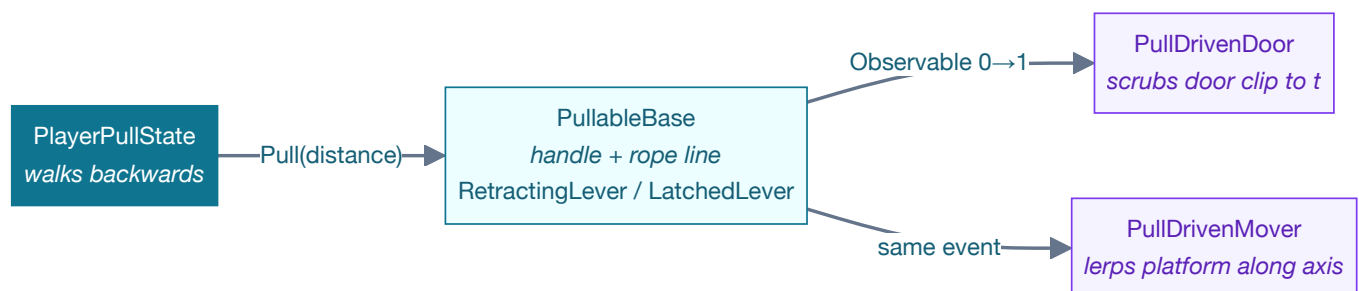


Fig. 4 — A lever never knows what it opens. It publishes normalized progress; any number of Dependents follow it, wired in the Inspector via `InterfaceRef`.

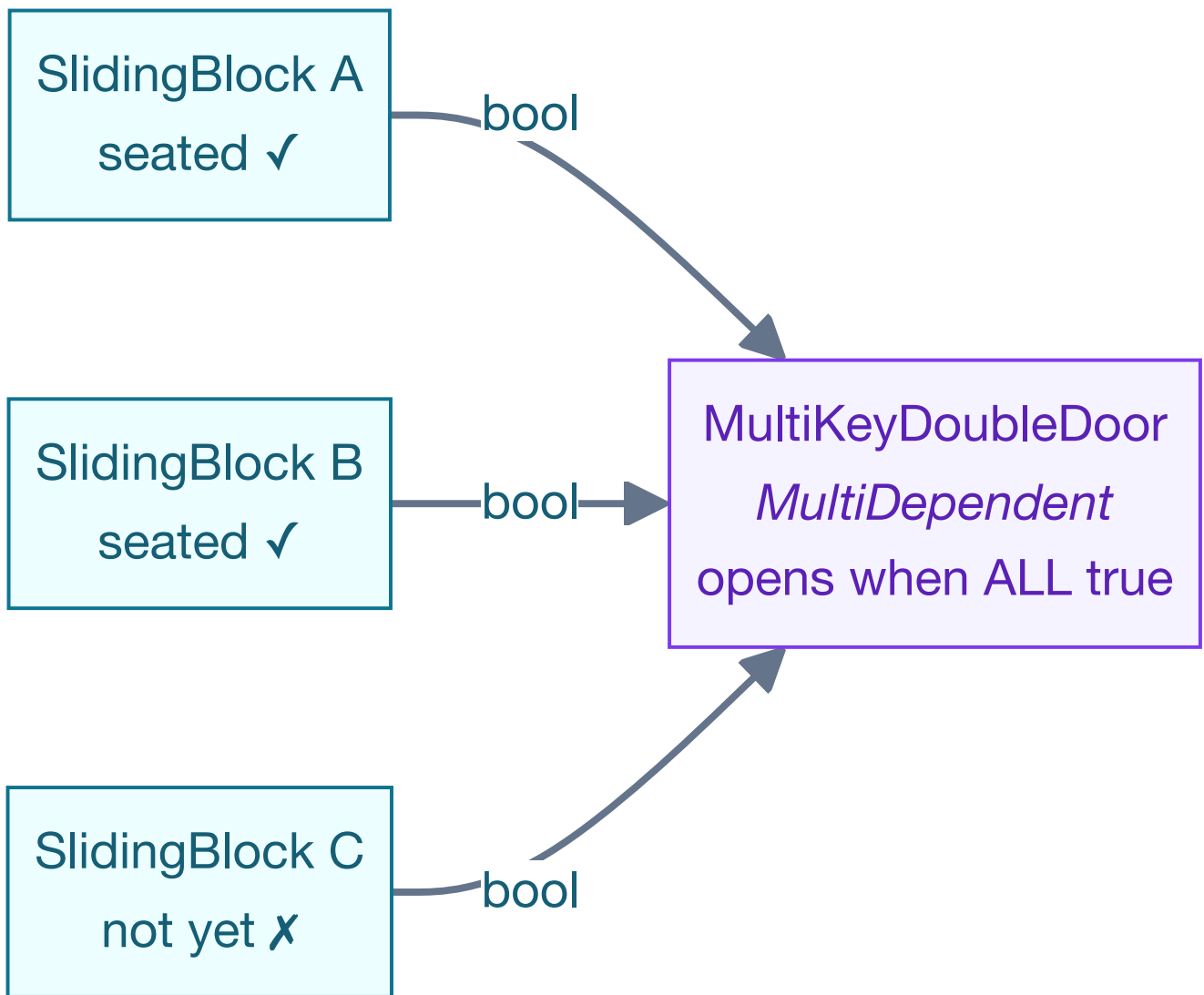


Fig. 5 — Multi-key puzzle door: a `MultiDependent<bool>` that re-evaluates on any change and opens once all sources report true.

4 Items System

The one rule: items end their own interactions. On `Use()` every item stores `context.EndInteraction` as its `ItemFinishedCallback` and invokes it from `PutAway()`. The state machine waits; the item decides when it's done — and can even hand back a new object to carry.

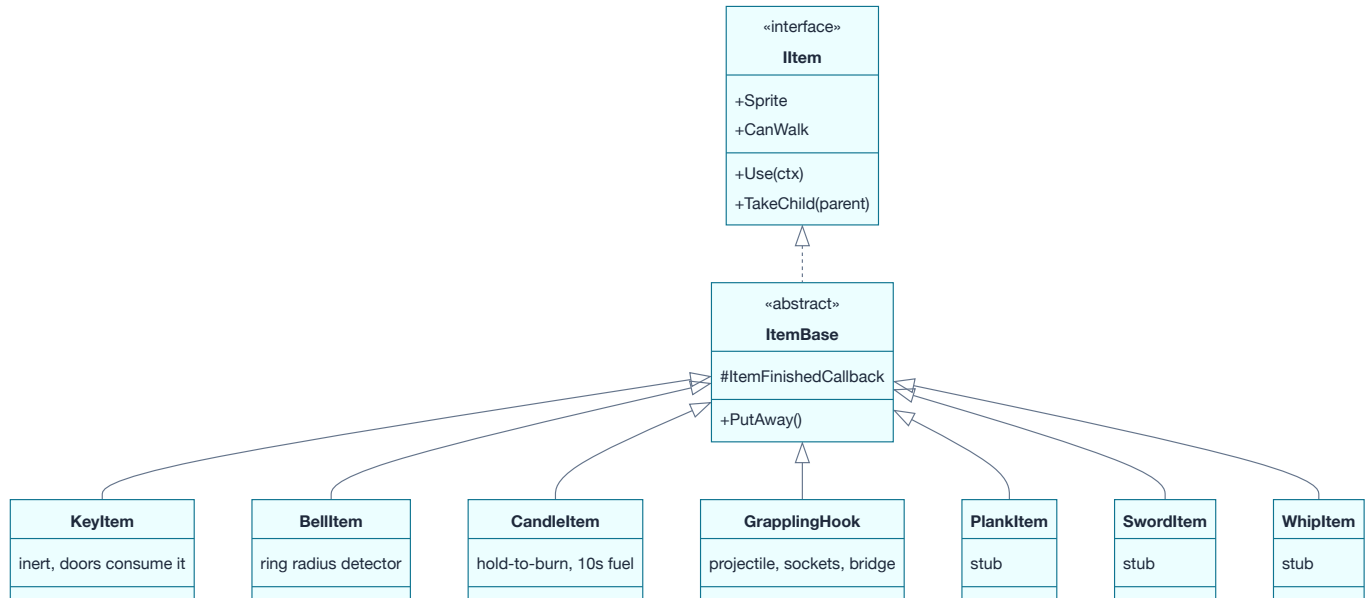


Fig. 6 — The item family. *PickupHolder* (an *Interactable*, *Kind* = *Equip*) is the world-side wrapper that gets items into the inventory.

► Grappling hook — three outcomes

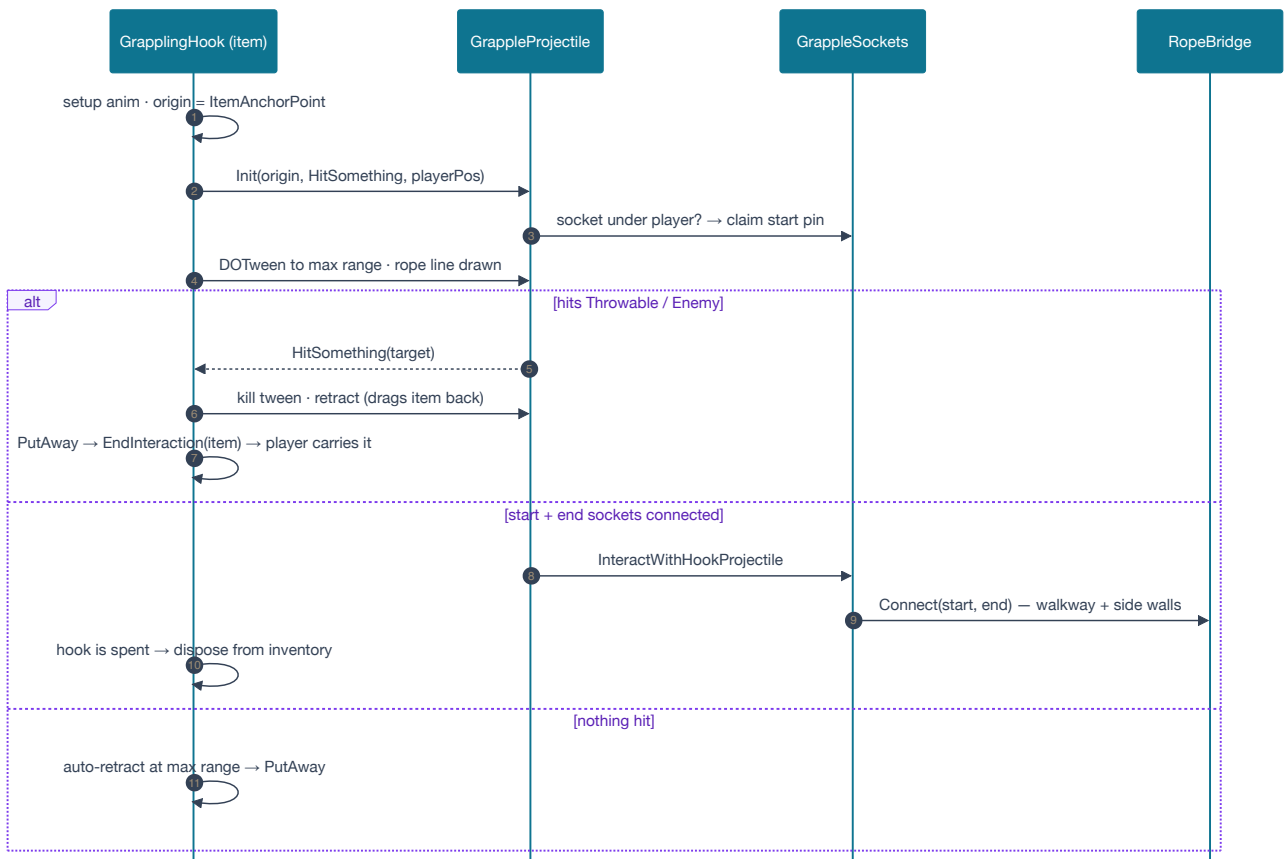


Fig. 7 — Grapple flow. RopeBridge builds side-wall colliders over Depth-layer pits and uses IgnoreCollision so the player can cross.

5 Overlap, Layers & Core Plumbing

► Overlap system — “what am I touching?”

One shared idea: keep a sprite-sized AABB (repositioned per look direction), run `Physics2D.OverlapAreaAll` with a layer mask, pick the collider with the **largest intersection area**. Several checkers are short-lived prefabs spawned just for one action.

Checker	Used by	Question it answers
<code>InteractableOverlap</code>	Player	Which <code>Interactable</code> am I facing? (feeds <code>Interact()</code>)
<code>KeyPortOverlap</code>	PushBlock, SlidableBase	Am I seated $\geq 60\%$ on a <code>KeyPort</code> ?
<code>MovementOverlap</code>	PullableBase	Can the lever handle move here? (boolean probe)
<code>DamageOverlap</code>	SlidableBase	Who's in a sliding block's path? (applies <code>IDamage</code>)
<code>GrappleTargetOverlap</code>	GrappleProjectile	What is the hook flying over?
<code>GrappleSocketOverlap</code>	GrappleProjectile	Is there an unused socket under the player / the hook?
<code>RopeObstructionOverlap</code>	RopeBridge	Which Depth-layer pits does the bridge span?

► Physics layers as state

Layer	Meaning
<code>Interactable</code>	Interact button can hit it (every <code>Interactable</code> self-assigns in <code>Awake</code>)
<code>Obstruction</code> / <code>Locked</code>	Blocks slides & pushes / seated blocks now permanent
<code>KeyPort</code>	Pressure-plate sockets — only port queries look here
<code>HookSocketUnused</code> → <code>Used</code>	Grapple socket claim state (flipped at runtime)
<code>Depth</code>	Pits/water the rope bridge spans
<code>TargetOverlap</code>	The checker objects themselves — excluded from masks to avoid self-hits

Names live in `GameUtilities` (plus the `KeyTypes` enum pairing keys to locks); indices are cached once in `LayerIndex`.

► Dependency plumbing & utilities

`IDependencySource<T>` family

`DependencySource<T>` (MonoBehaviour base), `Observable<T>` (plain C#, composed into classes that already extend `Interactable`), `Dependent<T>` (one source) and `MultiDependent<T>` (many) handle subscribe / initial-sync / unsubscribe.

`InterfaceRef<T>` + drawer

Serializable interface reference with an Inspector drawer that validates the dragged object. This makes puzzle wiring **designer-facing**, not code-facing.

Damage (minimal)

`IDamage` markers (`DamageOverlap` , `ExplosionDamageArea` , `GrappleProjectile`) → `IDamageable.ApplyDamage()` — currently only `EnemyDummy` . Player health not yet wired.

Utilities

`DepthSorter` (y-sorting each `LateUpdate`), `GridSnapper` (edit-mode tile snap), `Singleton<T>` , `DebugDraw` gizmos, `SpritePivotValidator` (pivots feed depth sorting).

► Animation: code-driven, awaitable

No transition graphs. C# “anim state” classes call `Animator.Play(hash)` directly. One-shots keep a **TimeSheet** (`Dictionary<hash, seconds>`) and `await Awaitable.WaitForSecondsAsync(...)` so gameplay sequences on animation completion. Directional clips are picked from `LookDirection` . World wrappers (`ChestOpenAnim` , door anims) are constructed by their owners; `PullDoorAnim` is the standout — animator speed pinned to 0 and the clip **scrubbed** to the lever's normalized progress.

6 End-to-End Walkthroughs

🔑 Opening a locked chest

- 1 Interact pressed → `InteractableOverlap` returns the `Chest` → idle dispatches on `Kind = Open`.
- 2 `OpenableBase.Interact` : current item is a matching `KeyItem` → consumed (`DisposeOfCurrentItem` , HUD updates).
- 3 Open animation plays; `SpawnItemOnOpen` instantiates the prize → `PickUpItem` ; if full, `ItemDropper` ejects the displaced item in a `PickupHolder`.
- 4 Back to idle.

🚪 Sliding-block puzzle opens a door

- 1 Kick a `BasicSlidingBlock` → path raycast; blocked → hurt-toe anim; clear → block tweens away, spawning its mover/target checkers.
- 2 Block stops at the obstruction; `KeyPortOverlap.FindKeyPort` finds it seated $\geq 60\%$ on a `SlidingKeyPort` → its `Observable<bool>` flips true and it moves to the `Locked` layer.
- 3 `MultiKeyDoubleDoor` re-evaluates; when **all** wired blocks report true, the door animation plays (Fig. 5).

🧶 Grappling a bomb across a gap

- 1 Use with `GrapplingHook` selected → setup anim → projectile flies from the `ItemAnchorPoint` , rope drawn behind it.
- 2 It overlaps a `Bomb` (a `ThrowableBase`) → callback to the gun → tween killed → retract drags the bomb back.
- 3 `PutAway` calls `EndInteraction(bomb)` → the player lands directly in `PlayerThrowState` , carrying a live bomb.

7 Known Rough Edges

Tracked in `REFACTOR_PLAN.md` — useful context when reading the code, not hidden bugs.

Vestigial files. `PlayerContext` , `PlayerAnimator` , `PlayerMover` (movement actually lives in `PlayerStateBase.Move`), empty `AnimClipBase` / `AnimStateBase` , the odd `IPlayerState` sketch, and the `Prototype/Apple*` learning FSM.

async void event-style methods in states and interactables (`SlideItem` , `CleanUp` , `Action`) — exceptions caught locally, but the pattern is slated for cleanup.

Overlap duplication. `DamageOverlap` / `MovementOverlap` re-implement the AABB + direction-helper logic that `OverlapCheckerBase` centralizes; offsets are hard-coded per prefab.

Stubs & gaps. `PlayerDeathState` , `TimedSlidingBlock.Timeout` , sword/whip/plank items, empty HUD methods, enemy AI. Damage never routes into `PlayerStatus` ; `ExplosionDamageArea` computes a falloff it never uses.